

Operating Systems: Process Management & Concurrency

Course: CS301 Operating Systems Architecture | **Author:** Dept. of Computer Science

1. Process Concept and Process Control Block (PCB)

A process is defined as a program in execution. It encompasses the executable code (text section), current activity tracked by the Program Counter (PC) and processor registers, a process stack containing temporary data (function parameters, return addresses, local variables), a data section containing global variables, and a heap for runtime dynamic memory allocation.

The Process Control Block (PCB) is the central operating system data structure used to maintain all information associated with an active process. Key fields inside a PCB include: Process State (New, Ready, Running, Waiting, Terminated), Program Counter indicating the next instruction, CPU registers (accumulators, index registers, stack pointers), CPU-scheduling information such as priority and queue pointers, Memory-management information (page tables, base and limit registers), and Accounting/I/O status information including open file descriptors.

2. Threads vs Processes

A thread is the smallest schedulable unit of CPU utilization, commonly termed a lightweight process. Threads belonging to the same process share their code section, data section, and OS resources (such as open files and signals). However, each thread maintains its own unique Thread ID, Program Counter, register set, and private execution stack.

Key benefits of multithreading include: High responsiveness in interactive applications, efficient resource sharing without costly IPC, economy due to lower context switching overhead, and scalability across multicore and multiprocessor hardware architectures.

3. Critical-Section Problem & Synchronization

A race condition occurs when multiple concurrent threads or processes access and manipulate shared data, and the final outcome depends on the arbitrary execution order. To guarantee deterministic execution, critical sections must be synchronized.

Any valid solution to the Critical-Section Problem must satisfy three mandatory requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other process can be executing in its critical section.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not in their remainder section can participate in deciding who enters next, and selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There must exist a bound on the number of times other processes are allowed to enter their critical sections after a process has requested entry and before that request is granted.

4. Semaphores and Mutex Locks

A Mutex Lock is a boolean synchronization lock: a process must acquire the lock before entering its critical section and release it upon departure. A Semaphore is an integer synchronization variable introduced by Edsger Dijkstra, manipulated strictly through two atomic operations: `wait()` and `signal()`.

Counting Semaphores range over an unrestricted domain to control access to finite resource pools. **Binary Semaphores** have an integer value constrained to 0 and 1, functioning identically to mutexes.

5. Deadlocks: The Four Necessary Conditions

A deadlock is a condition in which a set of concurrent processes are permanently blocked because each process holds a resource and waits for another resource held by another process in the set.

Coffman et al. established that a deadlock can arise if and only if the four following conditions hold simultaneously:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
2. **Hold and Wait:** A process must currently hold at least one resource and request additional resources held by other processes.
3. **No Preemption:** Resources cannot be forcibly seized; they can only be voluntarily released by the holding process.
4. **Circular Wait:** A closed chain of processes exists where each process holds resources required by the subsequent process in the loop.

Handling Strategies: Deadlock Prevention (invalidating at least one of the 4 conditions), Deadlock Avoidance (e.g. Dijkstra's Banker's Algorithm ensuring system always remains in a safe state), and Deadlock Detection with recovery.